TRAINING & PLACEMENT CELL – BHGCET

Email: tpo@gardividyapith.ac.in

WEEK 3: ARRAYS, SEARCHING & BASIC SORTING
(DSA FUNDAMENTALS)

Submission Report (Python)

Student Name	
Roll Number	
Branch	
Semester	
Submission Date	

Table of Contents

1. L – Learn (DSA Fundamentals)

1.1 Arrays (Core Foundation)

- **What is an array:** A collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together.
- **Indexing (0-based indexing):** In most programming languages, including Python, array elements are indexed starting from 0. The first element is at index 0, the second at index 1, and so on.
- **Types:**
  - *Static Arrays:* Have a fixed size defined at the time of creation.
  - *Dynamic Arrays:* Can grow or shrink in size dynamically. *Note: Python lists are implemented as dynamic arrays.*
- **Basic Operations & Time Complexity:**
  - *Traversal:* Visiting every element sequentially. O(n)
  - *Insertion:* Adding an element. O(n) worst-case (shifting elements).
  - *Deletion:* Removing an element. O(n) worst-case (shifting elements).
  - *Access:* Direct access via index. O(1)

1.2 Searching Algorithms

- **Linear Search:**
  - *Concept:* Sequentially checks each element of the array until a match is found or the whole array has been searched.
  - *Time Complexity:* O(n)
- **Binary Search:**
  - *Requirement:* The array **must be sorted**.
  - *Concept:* Repeatedly divides the search interval in half. If the value of the search key is less than the item in the middle of the interval, narrows the interval to the lower half. Otherwise, narrows it to the upper half.
  - *Time Complexity:* O(log n)

1.3 Basic Sorting Algorithms

- **Bubble Sort:**
  - *Concept:* Repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order.
  - *Time Complexity:* $O(n^2)$
- **Selection Sort:**
  - *Concept:* Divides the input list into two parts: a sorted sublist and an unsorted sublist. Repeatedly finds the minimum element from the unsorted sublist and moves it to the end of the sorted sublist.
  - *Time Complexity:* $O(n^2)$

- **Importance of Sorting:** Sorting data significantly optimizes search operations (enabling Binary Search), simplifies finding extreme values (min/max), and makes data readable and analyzable.

2. A – Apply (Implementations)

Note: Python lists are inherently dynamic. However, for these DSA fundamental exercises, we are utilizing them to demonstrate standard array-like operations.

Task 1: Array Operations

File Name: `array_operations.py`

Demonstrates traversing, inserting, and deleting elements from an array using Python lists.

```
# array_operations.py

def display_array(arr):
    print("Array elements:", end=" ")
    for item in arr:  # Traversal
        print(item, end=" ")
    print()

def main():
    # Initializing an array (Python list)
    my_array = [10, 20, 30, 40, 50]
    print("Initial Array:")
    display_array(my_array)

    # Inserting element 25 at index 2
    position = 2
    element = 25
    my_array.insert(position, element)
    print(f"\nAfter inserting {element} at index {position}:")
    display_array(my_array)

    # Deleting element at index 4
    del_position = 4
    removed_element = my_array.pop(del_position)
        print(f"\nAfter deleting element at index {del_position} (Value:
{removed_element}):")
    display_array(my_array)

if __name__ == "__main__":
    main()
```

Sample Output:

```
Initial Array:
Array elements: 10 20 30 40 50

After inserting 25 at index 2:
```

```
Array elements: 10 20 25 30 40 50

After deleting element at index 4 (Value: 40):
Array elements: 10 20 25 30 50
```

Task 2: Searching Implementation

File Name: `searching_algorithms.py`

Implements both Linear Search and Binary Search, testing for elements that exist and elements that do not exist.

```
# searching_algorithms.py

def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1

def binary_search(arr, target):
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

def main():
    array = [5, 12, 23, 34, 45, 56, 67, 78]
    print(f"Array for searching: {array}")

    print("\n--- Linear Search ---")
    target1 = 23
    idx1 = linear_search(array, target1)
    print(f"Searching for {target1}: {'Found at index ' + str(idx1) if idx1 !=
-1 else 'Not Found'}")

    target2 = 99
    idx2 = linear_search(array, target2)
    print(f"Searching for {target2}: {'Found at index ' + str(idx2) if idx2 !=
-1 else 'Not Found'}")

    print("\n--- Binary Search (Array must be sorted) ---")
```

```
    target3 = 67
    idx3 = binary_search(array, target3)
    print(f"Searching for {target3}: {'Found at index ' + str(idx3) if idx3 !=
-1 else 'Not Found'}")

    target4 = 10
    idx4 = binary_search(array, target4)
    print(f"Searching for {target4}: {'Found at index ' + str(idx4) if idx4 !=
-1 else 'Not Found'}")

if __name__ == "__main__":
    main()
```

Sample Output:

```
Array for searching: [5, 12, 23, 34, 45, 56, 67, 78]

--- Linear Search ---
Searching for 23: Found at index 2
Searching for 99: Not Found

--- Binary Search (Array must be sorted) ---
Searching for 67: Found at index 6
Searching for 10: Not Found
```

Task 3: Sorting Implementation

File Name: `sorting_algorithms.py`

Implements Bubble Sort and Selection Sort, displaying the array before and after sorting.

```
# sorting_algorithms.py

def bubble_sort(arr):
    n = len(arr)
    # Create a copy to avoid modifying original immediately
    sorted_arr = arr.copy()
    for i in range(n):
        for j in range(0, n - i - 1):
            if sorted_arr[j] > sorted_arr[j + 1]:
                # Swap elements
                      sorted_arr[j], sorted_arr[j + 1] = sorted_arr[j + 1],
sorted_arr[j]
    return sorted_arr

def selection_sort(arr):
    n = len(arr)
    sorted_arr = arr.copy()
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if sorted_arr[j] < sorted_arr[min_idx]:
                min_idx = j
        # Swap the found minimum element with the first element
                  sorted_arr[i], sorted_arr[min_idx] = sorted_arr[min_idx],
sorted_arr[i]
    return sorted_arr

def main():
    array1 = [64, 34, 25, 12, 22, 11, 90]
    print("--- Bubble Sort ---")
    print(f"Original array: {array1}")
    print(f"Sorted array:   {bubble_sort(array1)}\n")

    array2 = [29, 10, 14, 37, 13]
    print("--- Selection Sort ---")
    print(f"Original array: {array2}")
    print(f"Sorted array:   {selection_sort(array2)}")

if __name__ == "__main__":
    main()
```

Sample Output:

```
--- Bubble Sort ---
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

--- Selection Sort ---
Original array: [29, 10, 14, 37, 13]
Sorted array: [10, 13, 14, 29, 37]
```

Task 4: Problem Solving

File Name: `problem_solving_arrays.py`

Solves 4 foundational array problems: max/min, reverse, frequency count, and second largest element.

```
# problem_solving_arrays.py

def find_max_min(arr):
    if not arr:
        return None, None
    max_val = min_val = arr[0]
    for num in arr:
        if num > max_val: max_val = num
        if num < min_val: min_val = num
    return max_val, min_val

def reverse_array(arr):
    # Using two-pointer approach for basic DSA understanding
    rev_arr = arr.copy()
    left, right = 0, len(rev_arr) - 1
    while left < right:
        rev_arr[left], rev_arr[right] = rev_arr[right], rev_arr[left]
        left += 1
        right -= 1
    return rev_arr

def count_frequency(arr):
    freq = {}
    for num in arr:
        freq[num] = freq.get(num, 0) + 1
    return freq

def find_second_largest(arr):
    if len(arr) < 2:
        return None
    largest = second_largest = float('-inf')
    for num in arr:
        if num > largest:
            second_largest = largest
            largest = num
```

```
        elif num > second_largest and num != largest:
            second_largest = num
    return second_largest if second_largest != float('-inf') else None

def main():
    arr = [12, 35, 1, 10, 34, 1]
    print(f"Array: {arr}\n")

    # 1. Find max and min
    mx, mn = find_max_min(arr)
    print(f"1. Maximum: {mx}, Minimum: {mn}")

    # 2. Reverse array
    print(f"2. Reversed Array: {reverse_array(arr)}")

    # 3. Frequency count
    freq_arr = [2, 3, 2, 5, 3, 3]
    print(f"3. Frequencies of {freq_arr}: {count_frequency(freq_arr)}")

    # 4. Second largest
    print(f"4. Second Largest in {arr}: {find_second_largest(arr)}")

if __name__ == "__main__":
    main()
```

Sample Output:

```
Array: [12, 35, 1, 10, 34, 1]

1. Maximum: 35, Minimum: 1
2. Reversed Array: [1, 34, 10, 1, 35, 12]
3. Frequencies of [2, 3, 2, 5, 3, 3]: {2: 2, 3: 3, 5: 1}
4. Second Largest in [12, 35, 1, 10, 34, 1]: 34
```

3. P – Prove (Submission Requirements)

The following items are fulfilled and documented within this report:

- ✅ Code files provided for array operations, searching algorithms, and sorting algorithms.
- ✅ Sample outputs successfully generated and attached below each code segment.
- ✅ Problem-solving solutions implemented for all 4 fundamental array tasks.
- ✅ Theoretical explanations provided in the next section.

4. Conceptual Explanations

4.1 Difference between Linear Search and Binary Search

Basis	Linear Search	Binary Search
Concept	Checks every element sequentially.	Divides search space in half recursively/iteratively.
Requirement	Works on both sorted and unsorted arrays.	Requires the array to be **sorted** prior to searching.
Time Complexity	O(n) - Linear Time	O(log n) - Logarithmic Time
Efficiency	Less efficient for large datasets.	Highly efficient for large datasets.

4.2 When to use Sorting

- **When fast retrieval is required:** Sorting allows the use of Binary Search (O(log n)) instead of Linear Search (O(n)).
- **When finding duplicates or frequencies:** Sorted data groups similar items together, making it easier to count or eliminate duplicates in O(n) time.
- **When solving extreme value problems:** Finding Kth largest/smallest elements becomes straightforward after sorting.
- **Data visualization and presentation:** Sorted datasets are naturally more readable and interpretable for end-users.

4.3 Time Complexity Summary

Algorithm / Operation	Best Case	Worst Case
Array Traversal	O(n)	O(n)
Array Insertion (at index)	O(1) *(at end)*	O(n) *(at start/mid)*
Array Deletion (at index)	O(1) *(at end)*	O(n) *(at start/mid)*

Algorithm / Operation	Best Case	Worst Case
Linear Search	O(1)	O(n)
Binary Search	O(1)	O(log n)
Bubble Sort	O(n) *(optimized)*	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$

5. References

Standard Books

- *Introduction to Algorithms* by Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.
- *Data Structures and Algorithms Made Easy* by Narasimha Karumanchi.
- *Algorithms* by Robert Sedgewick and Kevin Wayne.

Online Practice Platforms

- **GeeksforGeeks:** https://www.geeksforgeeks.org/
- **HackerRank:** https://www.hackerrank.com/

6. Declaration

I hereby declare that the implementations and conceptual explanations presented in this submission report are my original work, tested successfully in Python, and align with the guidelines provided by the Training & Placement Cell.

Signature of the Student	Signature of the Faculty
Date: _____	Date: _____